

Exception

Ch 14

Object Oriented Programming
CSE-4301

Presented by

Rafsanjany Kushol
Assistant Professor



Exception

- ✓ Provide a systematic, object-oriented approach to handle errors generated by C++ classes.
- ✓ Exceptions are errors that occur at runtime.
- ✓ They are caused by a wide variety of exceptional circumstance, such as:
 - running out of memory,
 - not being able to open a file,
 - trying to initialize an object to an impossible value, or
 - using an out-of-bounds index to a vector.

Why Do We Need Exceptions?

- ✓ Check each time when the function is called.
- ✓ Requires lots of code and make code hard to read
- ✓ In a class definition errors may take place without a function being explicitly called.
 - Constructor is called implicitly so if else check is not possible

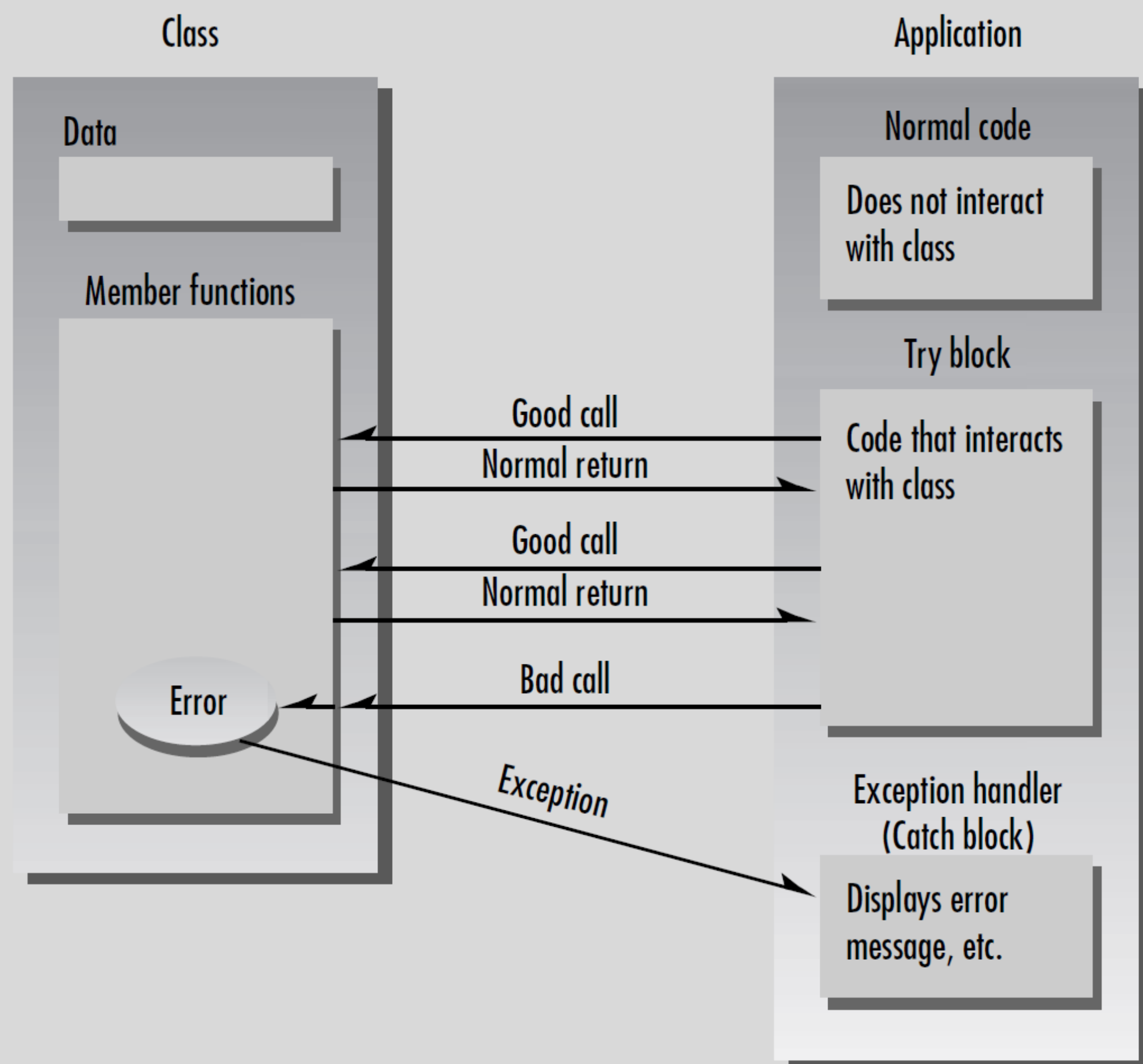
```
if( somefunc() == ERROR_RETURN_VALUE )
    //handle the error or call error-handler function
else
    //proceed normally
if( anotherfunc() == NULL )
    //handle the error or call error-handler function
else
    //proceed normally
if( thirdfunc() == 0 )
    //handle the error or call error-handler function
else
    //proceed normally
```

Exception Mechanism

The exception mechanism uses three new C++ keywords:

- throw
- catch
- try

Also, a kind of entity called an exception class needs to be created



Exception Syntax

- ✓ Application makes a mistake -> causing an error to be detected in a member function.
- ✓ This member function then informs the application that an error has occurred. When exceptions are used, this is called **throwing** an exception.
- ✓ In the application, we install a separate section of code to handle the error. This code is called an **exception handler** or **catch block**; it *catches* the exceptions thrown by the member function.
- ✓ Any code in the application that uses objects of the class is enclosed in a **try block**. Errors generated in the **try** block will be caught in the **catch** block.

Exception Mechanism

```

class AClass                                // a class
{
public:
    class AnError                            // exception class
    {
    };
    void Func()                             // a member function
    {
        if( /* error condition */ )
            throw AnError();                // throw exception
    }
};

int main()                                  // application
{
    try                                     // try block
    {
        AClass obj1;                       // interact with AClass objects
        obj1.Func();                       // may cause error
    }
    catch (AClass::AnError)                 // exception handler
    {                                       // (catch block)
    }
    return 0;
}

```

Exception Example

```
#include <iostream>
using namespace std;
const int MAX = 3;                                //stack holds 3 ints
////////////////////////////////////
class Stack
{
private:
    int st[MAX];                                    //stack: array of integers
    int top;                                         //index of top of stack
public:
    class Range                                     ///exception class for Stack
    {
        //note: empty class body
    };
//-----
    Stack()                                         //constructor
    {
        top = -1;
    }
}
```

Exception Example

```

void push(int var)
{
    if(top >= MAX-1)           ///if stack full,
        throw Range();        ///throw exception
    st[++top] = var;           ///put number on stack
}

//-----

int pop()
{
    if(top < 0)                ///if stack empty,
        throw Range();        ///throw exception
    return st[top--];          ///take number off stack
}

};

```

Exception Example

```
int main()
{
    Stack s1;

    try
    {
        s1.push(11);
        s1.push(22);
        s1.push(33);
//      s1.push(44);                                //oops: stack full
        cout << "1: " << s1.pop() << endl;
        cout << "2: " << s1.pop() << endl;
        cout << "3: " << s1.pop() << endl;
        cout << "4: " << s1.pop() << endl;    //oops: stack empty
    }
```

Exception Example

```
catch(Stack::Range)                                //exception handler
{
    cout << "Exception: Stack Full or Empty" << endl;
}

cout << "Arrive here after catch (or normal exit)" << endl;
return 0;
}
```

Multiple Exceptions

```

class Stack
{
private:
    int st[MAX];           //stack: array of integers
    int top;               //index of top of stack
public:
    class Full { };        //exception class
    class Empty { };       //exception class
//-----
    Stack()                //constructor
    {
        top = -1;
    }
//-----
    void push(int var)      //put number on stack
    {
        if(top >= MAX-1)    //if stack full,
            throw Full();   //throw Full exception
        st[++top] = var;
    }
//-----
    int pop()              //take number off stack
    {
        if(top < 0)         //if stack empty,
            throw Empty();   //throw Empty exception
        return st[top--];
    }
};

```

Multiple Exceptions

```
int main()
{
    Stack s1;

    try {
        s1.push(11);
        s1.push(22);
        s1.push(33);
        //s1.push(44); //oops: stack full
        cout << "1: " << s1.pop() << endl;
        cout << "2: " << s1.pop() << endl;
        cout << "3: " << s1.pop() << endl;
        cout << "4: " << s1.pop() << endl; //oops: stack empty
    }
    catch (Stack::Full) {
        cout << "Exception: Stack Full" << endl;
    }
    catch (Stack::Empty)
    {
        cout << "Exception: Stack Empty DONOT POP" << endl;
    }
}
```

Exception Another Example

```

class Distance                                //English Distance class
{
private:
    int feet;
    float inches;
public:
    class InchesEx { };                      //exception class
//-----
    Distance()                               //constructor (no args)
    {
        feet = 0;
        inches = 0.0;
    }
//-----
    Distance(int ft, float in)               //constructor (two args)
    {
        if(in >= 12.0)                       //if inches too big,
            throw InchesEx();                //throw exception
        feet = ft;
        inches = in;
    }

```

Exception Another Example

```
void getdist()                //get length from user
{
    cout << "\nEnter feet: ";
    cin >> feet;
    cout << "Enter inches: ";
    cin >> inches;
    if(inches >= 12.0)         //if inches too big,
        throw InchesEx();    //throw exception
}
```

```
void showdist()              //display distance
{
    cout << feet << "'-" << inches << "'\"";
}
~Distance()
{
    cout<<"Dest~ called\n";
}
```

Exception Another Example

```
int main()
{
    Distance dist2;
    try {
        Distance dist1(17, 3.5);    //2-arg constructor
        //Distance dist2;           //no-arg constructor
        dist2.getdist();            //get distance
        //display distances
        cout << "\ndist1 = ";
        dist1.showdist();
        cout << "\ndist2 = ";
        dist2.showdist();
    } catch (Distance::InchesEx) { //catch exceptions
        cout << "\nInitialization error: "
             << "inches value is too large.";
    }
    cout << endl;
```

Exceptions with Arguments

```

class Distance                                //English Distance class
{
private:
    int feet;
    float inches;
public:
//-----
    class InchesEx                            //exception class
    {
public:
        string origin;                       //for name of routine
        float iValue;                       //for faulty inches value

        InchesEx(string orr, float in)      //2-arg constructor
        {
            origin = orr;                   //store string
            iValue = in;                    //store inches
        }
    };                                       //end of exception class
//-----
    Distance()                               //constructor (no args)
    {
        feet = 0;
        inches = 0.0;
    }

```

Exceptions with Arguments

```
Distance(int ft, float in)    //constructor (two args)
{
    if(in >= 12.0)
        throw InchesEx("2-arg constructor", in);
    feet = ft;
    inches = in;
}
```

```
void getdist()                //get length from user
{
    cout << "\nEnter feet: ";
    cin >> feet;
    cout << "Enter inches: ";
    cin >> inches;
    if(inches >= 12.0)
        throw InchesEx("getdist() function", inches);
}
```

```
void showdist()               //display distance
{
    cout << feet << "'-" << inches << "'";
}
```

Exceptions with Arguments

```

int main()
{
    Distance dist1(17, 13.5);
    try {
        Distance dist2;
        dist2.getdist();
        //display distances
        cout << "\ndist1 = ";
        dist1.showdist();
        cout << "\ndist2 = ";
        dist2.showdist();
    } catch (Distance::InchesEx ix) {
        cout << "\nInitialization error in " << ix.origin
              << ".\n    Inches value of " << ix.iValue
              << " is too large.";
    }
}

```

Enter feet: 7
 / Enter inches: 13.5

Initialization error in getdist() function.
 Inches value of 13.5 is too large.
 //get value

Distance dist1(17, 22.25);

the resulting exception will cause this error message:

Initialization error in 2-arg constructor.
 Inches value of 22.25 is too large.

The bad_alloc Class

```
int main()
{
    const unsigned long SIZE = 10000;    //memory size
    char* ptr;                          //pointer to memory

    try
    {
        ptr = new char[SIZE];            //allocate SIZE bytes
    }
    catch(bad_alloc)                    //exception handler
    {
        cout << "\nbad_alloc exception: can't allocate memory.\n";
        return(1);
    }
    delete[] ptr;                      //deallocate memory
    cout << "\nMemory use is successful.\n";
    return 0;
}
```

Exception Notes

- ✓ Not for Every Situation
 - impose a certain overhead in terms of program size and (when an exception occurs) time.

- ✓ Destructors Called Automatically
 - When an exception is thrown, a destructor is called automatically for any object that was created by the code up to that point in the try block.

Thank You

Any questions?

kushol@iut-dhaka.edu

